实验 3 HH 模型、LIF 模型与神经计算建模

3.1 实验目的

1. **了解 HH 模型的基本作用：** 了解 Hodgkin-Huxley 模型如何描述动作电位的产生过程，以及它与简化神经元模型的区别。

2. **掌握 LIF 模型的基本建模方法：** 理解 LIF 模型的动力学方程，并能够用数值仿真的方式观察膜电位随时间的变化。

3. **分析不同输入对神经元发放的影响：** 观察恒定电流输入和周期电流输入下，神经元膜电位轨迹和脉冲发放模式的差异。

4. **比较理论结果与仿真结果：** 利用课上给出的 LIF 发放周期与发放频率公式，将理论结果与数值仿真结果进行对照分析。

3.2 实验说明

本次实验围绕本周课程中讲到的 `HH` 模型 和 `LIF` 模型 展开。与实验 2 类似，本次实验仍然以“观察现象、修改参数、分析结果”为主，不要求同学们从零开始实现完整复杂模型。

本次实验中，主线部分一共设置了 3 **处需要自行补全的** TODO：

1. 在 LIF 恒定电流仿真中补全膜电位更新公式；

2. 在 LIF 恒定电流仿真中补全超过阈值后的发放与复位逻辑；

3. 在发放频率对照部分补全理论频率函数。

在开始实验之前，请先激活实验 1 中创建的虚拟环境，并安装本次实验需要用到的库（已经安装可以跳过）：

```
conda activate brain
pip install numpy matplotlib
```

如果校园网下载速度较慢，也可以使用镜像源，例如：

```
pip install numpy matplotlib -i https://pypi.tuna.tsinghua.edu.cn/simple
```

一、HH 模型现象观察

Hodgkin-Huxley（HH）模型是神经科学中非常经典的生物神经元模型。它与我们前面接触过的 M-P 神经元、人工神经元和简化脉冲神经元不同，HH 模型会显式描述膜电位以及离子通道开闭变量的变化，因此能够更真实地刻画动作电位的产生过程。

在本实验中，HH 模型的任务定位是“观察现象”，而不是要求大家完整推导所有公式。为了让计算过程更直观，下面不使用现成的微分方程求解器，而是采用**显式欧拉法**按时间步推进膜电位和门控变量。

这里需要特别注意：`dt` 表示数值积分时使用的时间步长，它是**数值计算参数**，不是生物学参数。`dt` 过大时，动作电位的峰值、波形甚至动作电位次数都可能发生变化，这类变化应理解为数值离散误差，不能直接当作模型本身的生物学现象来解释。本实验默认建议使用较小的时间步长，例如 `dt = 0.01 ms`；只有在专门比较数值误差时，才建议把它适当调大。

运行下面代码，重点观察：

- 外加电流开始和结束时刻；
- 膜电位是否出现动作电位峰值；
- 改变刺激强度后，动作电位次数是否变化。

> 注意：下面 HH 图中的灰色虚线只是 `0 mV` 参考线，用来帮助观察膜电位是否越过 `0 mV`；它**不是发放阈值线**。后面 LIF 图中的虚线才表示模型设定的阈值。

```python
import numpy as np
import matplotlib.pyplot as plt

# HH 模型常用参数（单位与经典模型一致）
C_m = 1.0
g_Na = 120.0
```

```python
g_K = 36.0
g_L = 0.3
E_Na = 50.0
E_K = -77.0
E_L = -54.387
V_rest = -65.0



# n 门控变量的开启速率
def alpha_n(V):
    x = V + 55.0
    # 当分母接近 0 时，直接使用极限值，避免数值不稳定
    return 0.01 * x / (1 - np.exp(-x / 10)) if abs(x) > 1e-6 else 0.1


# n 门控变量的关闭速率
def beta_n(V):
    return 0.125 * np.exp(-(V + 65.0) / 80)


# m 门控变量的开启速率
def alpha_m(V):
    x = V + 40.0
    return 0.1 * x / (1 - np.exp(-x / 10)) if abs(x) > 1e-6 else 1.0


# m 门控变量的关闭速率
def beta_m(V):
    return 4.0 * np.exp(-(V + 65.0) / 18)


# h 门控变量的开启速率
def alpha_h(V):
    return 0.07 * np.exp(-(V + 65.0) / 20)


# h 门控变量的关闭速率
def beta_h(V):
    return 1.0 / (1 + np.exp(-(V + 35.0) / 10))


# 定义外加电流随时间的变化方式
def external_current(t, amplitude=10.0, start=10.0, end=40.0):
    # 在指定时间窗内施加一个矩形电流脉冲
    return amplitude if start <= t <= end else 0.0
```

```python
# 本实验要求修改的参数
stim_amplitude = 5.0    # 外加电流幅值，决定刺激强弱
stim_start = 10.0       # 外加电流开始时刻（ms）
stim_end = 40.0         # 外加电流结束时刻（ms）
dt = 0.01               # 数值积分时间步长（ms）

# 其余默认参数（一般不需要修改）
time_window = 60.0      # 总仿真时长（ms）

# 在静息电位处，使用稳态值作为初始门控变量
m0 = alpha_m(V_rest) / (alpha_m(V_rest) + beta_m(V_rest))
h0 = alpha_h(V_rest) / (alpha_h(V_rest) + beta_h(V_rest))
n0 = alpha_n(V_rest) / (alpha_n(V_rest) + beta_n(V_rest))

time = np.arange(0, time_window + dt, dt)
current = np.array([external_current(t, stim_amplitude, stim_start, sti
m_end) for t in time])
voltage = np.zeros_like(time)
m_values = np.zeros_like(time)
h_values = np.zeros_like(time)
n_values = np.zeros_like(time)

# 设置初始状态
voltage[0] = V_rest
m_values[0] = m0
h_values[0] = h0
n_values[0] = n0

for i in range(1, len(time)):
    V_prev = voltage[i - 1]
    m_prev = m_values[i - 1]
    h_prev = h_values[i - 1]
    n_prev = n_values[i - 1]
    I_ext = current[i - 1]

    I_Na = g_Na * (m_prev ** 3) * h_prev * (V_prev - E_Na)
    I_K = g_K * (n_prev ** 4) * (V_prev - E_K)
```

```
    I_L = g_L * (V_prev - E_L)

    # 先计算各变量的导数
    dVdt = (I_ext - I_Na - I_K - I_L) / C_m
    dmdt = alpha_m(V_prev) * (1 - m_prev) - beta_m(V_prev) * m_prev
    dhdt = alpha_h(V_prev) * (1 - h_prev) - beta_h(V_prev) * h_prev
    dndt = alpha_n(V_prev) * (1 - n_prev) - beta_n(V_prev) * n_prev

    # 再用欧拉法更新到下一时刻
    voltage[i] = V_prev + dt * dVdt
    m_values[i] = m_prev + dt * dmdt
    h_values[i] = h_prev + dt * dhdt
    n_values[i] = n_prev + dt * dndt

fig, axes = plt.subplots(2, 1, figsize=(10, 7), sharex=True)

axes[0].plot(time, current, color="tab:orange")
axes[0].set_title("Input Current of HH Model")
axes[0].set_ylabel("I_ext")
axes[0].grid(alpha=0.3)

axes[1].plot(time, voltage, color="tab:blue")
axes[1].axhline(0.0, color="gray", linestyle="--", alpha=0.7, label="0
mV reference")
axes[1].set_title("Membrane Potential of HH Model")
axes[1].set_xlabel("Time (ms)")
axes[1].set_ylabel("V_m (mV)")
axes[1].grid(alpha=0.3)
axes[1].legend()

plt.tight_layout()
plt.show()
```

修改以下参数：

- stim_amplitude
- stim_start
- stim_end
- dt

其中，dt 只建议在较小范围内调整，例如 0.01 ~ 0.02 ms；如果把 dt 调得过大，

图像中出现的差异主要反映数值误差，而不是 HH 模型本身的生物学差异。

完成以下现象观察，并在报告中分析结果：

1. 改变 `stim_amplitude` 后，膜电位是否会从“没有明显动作电位”变成“出现明显动作电位”？
2. 改变刺激持续时间后，动作电位的次数是否变化？
3. 若改变 `dt` 导致波形或动作电位次数变化，应如何判断这种变化是数值误差还是模型本身的生物学差异？
4. 与后面要做的 LIF 模型相比，HH 模型能描述哪些更细致的生物学现象？

二、LIF 模型：恒定电流输入下的膜电位与发放

LIF（Leaky Integrate-and-Fire）模型是对生物神经元的简化。它把复杂的离子通道过程进行了整体化处理，只保留最核心的几个机制：

- 电流输入会使膜电位上升；
- 漏电项会使膜电位逐渐回到静息值；
- 当膜电位达到阈值时，神经元发放一个脉冲；
- 发放后膜电位复位。

课上已经给出了 LIF 模型的标准形式：

$$\tau_m \frac{du}{dt} = -(u(t) - u_{rest}) + RI(t)$$

为了把连续时间微分方程写成程序中的离散更新形式，可以先做如下整理：

两边同时除以 τ_m，得到

$$\frac{du}{dt} = \frac{-(u(t) - u_{rest}) + RI(t)}{\tau_m}$$

再使用欧拉法近似：

$$u(t+\Delta t) \approx u(t) + \Delta t \cdot \frac{du}{dt}$$

将上式代入，可得离散更新公式：

$$u(t+\Delta t) \approx u(t) + \frac{\Delta t}{\tau_m}\left(-(u(t) - u_{rest}) + RI(t)\right)$$

这就是下面 TODO 1 中需要补全的核心更新关系。

下面先做恒定电流输入下的 LIF 仿真。代码里留出了 TODO 1 和 TODO 2，请你根据上述公式补全膜电位更新以及超过阈值后的处理逻辑。

```
import numpy as np
import matplotlib.pyplot as plt



# 根据 LIF 动力学方程更新一个时间步的膜电位
def lif_step(v_prev, input_current, tau_m, dt, R=1.0, v_rest=0.0):
    # TODO 1:
    # 请根据上述公式补全下面这一行，计算下一个时间步的膜电位
    # 不会的可以提问助教
    raise NotImplementedError("请补全 lif_step 函数中的膜电位更新公式")


# 在恒定电流输入下仿真 LIF 神经元的膜电位与发放情况
def simulate_lif_constant(
    current_value,
    tau_m=10.0,
    v_rest=0.0,
    v_reset=0.0,
    v_th=15.0,
    R=1.0,
    dt=0.05,
    time_window=100.0,
):
    time = np.arange(0, time_window + dt, dt)
    current = np.full_like(time, current_value)
    voltage = np.zeros_like(time)
    spikes = np.zeros_like(time)


    v = v_rest
```

```python
    for i in range(1, len(time)):
        # 先根据 LIF 动力学方程更新膜电位
        v = lif_step(v, current[i], tau_m, dt, R, v_rest)

        # TODO 2：请补全阈值判断逻辑
        # 提示：
        # 1. 需要分别考虑“达到阈值”和“未达到阈值”两种情况
        # 2. 达到阈值时，既要体现发放事件，也要处理图上的电压显示和后续状态
        # 3. 未达到阈值时，应保留当前时刻更新后的膜电位
        raise NotImplementedError("请补全超过阈值后的发放与复位逻辑")

    return time, current, voltage, spikes


# 本实验要求修改的参数
I_const = 20.0          # 恒定输入电流大小
tau_m = 10.0            # 膜时间常数，决定膜电位变化快慢
V_th = 15.0             # 发放阈值
R = 1.0                 # 膜电阻
dt = 0.05               # 数值仿真的时间步长（ms）

# 其余默认参数（一般不需要修改）
V_rest = 0.0            # 静息电位
V_reset = 0.0           # 发放后的复位电位
time_window = 100.0     # 总仿真时长（ms）

time, current, voltage, spikes = simulate_lif_constant(
    current_value=I_const,
    tau_m=tau_m,
    v_rest=V_rest,
    v_reset=V_reset,
    v_th=V_th,
    R=R,
    dt=dt,
    time_window=time_window,
```

```
)

fig, axes = plt.subplots(2, 1, figsize=(10, 7), sharex=True)

axes[0].plot(time, current, color="tab:orange")
axes[0].set_title("Constant Input Current")
axes[0].set_ylabel("I(t)")
axes[0].grid(alpha=0.3)

axes[1].plot(time, voltage, color="tab:blue", label="membrane potential
")
axes[1].axhline(V_th, color="tab:red", linestyle="--", label="threshold
")
axes[1].set_title("LIF Membrane Potential under Constant Input")
axes[1].set_xlabel("Time (ms)")
axes[1].set_ylabel("V_m (mV)")
axes[1].grid(alpha=0.3)
axes[1].legend()

plt.tight_layout()
plt.show()
```

请先补全 `lif_step` 和阈值判断部分，再运行整个脚本。

补全 `lif_step` 和阈值判断部分后，再修改以下参数：

- `I_const`
- `tau_m`
- `V_th`
- `R`
- `dt`

在报告中分析以下问题：

1. 增大输入电流 `I_const` 后，膜电位上升速度和发放节律如何变化？
2. 增大阈值 `V_th` 后，神经元为什么更难发放？
3. 时间常数 `tau_m` 变大或变小时，膜电位积累过程有什么变化？

三、LIF 模型：理论发放频率与仿真结果对照

对于恒定电流输入，课上已经给出了 LIF 模型的发放周期公式：

$$T = \tau_m \ln \frac{RI}{RI - V_{th}}$$

注意，这里假设静息电位 $V_{rest} = 0$ 且复位电位 $V_{reset} = 0$

如果时间单位是毫秒，那么平均发放频率可以写成：

$$f = \frac{1000}{T}$$

当 `RI <= V_th` 时，膜电位达不到阈值，因此不会发放脉冲，频率记为 `0`。

下面给出理论结果与仿真结果的对照代码。这里保留 TODO 3，请你补全理论频率函数。

```
import numpy as np
import matplotlib.pyplot as plt



# 根据 LIF 动力学方程更新一个时间步的膜电位
def lif_step(v_prev, input_current, tau_m, dt, R=1.0, v_rest=0.0):
    return v_prev + dt / tau_m * (-(v_prev - v_rest) + R * input_curren
t)


# 在恒定电流输入下仿真 LIF 神经元，并返回平均发放频率
def simulate_lif_constant(
    current_value,
    tau_m=10.0,
    v_rest=0.0,
    v_reset=0.0,
    v_th=15.0,
    R=1.0,
    dt=0.02,
    time_window=1000.0,
):
    time = np.arange(0, time_window + dt, dt)
    spikes = np.zeros_like(time)
    v = v_rest


    for i in range(1, len(time)):
        v = lif_step(v, current_value, tau_m, dt, R, v_rest)
```

```
        if v >= v_th:
            spikes[i] = 1
            v = v_reset


    # 这里把时间窗从 ms 转换为 s，再计算平均发放频率
    firing_rate = spikes.sum() / (time_window / 1000.0)
    return firing_rate


# 根据课上理论公式计算 LIF 神经元的理论发放频率
def theoretical_firing_rate(I, tau_m=10.0, v_th=15.0, R=1.0):
    # TODO 3:
    # 请根据课上公式补全理论发放频率函数
    # 提示:
    # 1. 先判断 R * I 是否小于等于 v_th
    # 2. 若不发放，则频率记为 0
    # 3. 若发放，则先算周期 T，再计算 f
    raise NotImplementedError("请补全 theoretical_firing_rate 函数")



# 本实验要求修改的参数
tau_m = 10.0                    # 膜时间常数
V_th = 15.0                     # 发放阈值
R = 1.0                         # 膜电阻
dt = 0.02                       # 仿真时间步长（ms）
I_values = np.arange(10, 31, 2) # 依次测试的一组输入电流值


# 其余默认参数（一般不需要修改）
time_window = 1000.0            # 用于统计平均发放频率的总时长（ms）


theory_rates = []
sim_rates = []


for I in I_values:
    theory_rates.append(theoretical_firing_rate(I, tau_m=tau_m, v_th=V_
th, R=R))
    sim_rates.append(
```

```
        simulate_lif_constant(
            current_value=I,
            tau_m=tau_m,
            v_th=V_th,
            R=R,
            dt=dt,
            time_window=time_window,
        )
    )


plt.figure(figsize=(8, 5))
plt.plot(I_values, theory_rates, marker="o", label="theory")
plt.plot(I_values, sim_rates, marker="s", label="simulation")
plt.xlabel("Input Current I")
plt.ylabel("Firing Rate (Hz)")
plt.title("Theoretical vs Simulated Firing Rate of LIF")
plt.grid(alpha=0.3)
plt.legend()
plt.show()
```

请先补全 `theoretical_firing_rate` 函数，再运行整个脚本。

补全 `theoretical_firing_rate` 函数后，再修改以下参数：

- `I_values`
- `tau_m`
- `V_th`
- `R`
- `dt`

在报告中分析以下问题：

1. 哪些输入电流区间不会触发神经元发放？为什么？
2. 理论曲线和仿真曲线是否一致？如果存在偏差，可能来自哪些原因？当时间步长 `dt` 改变后，二者的贴合程度是否变化？

四、LIF 模型：周期电流输入

除了恒定电流输入，课上和参考文件中还讨论了周期电流输入对 LIF 模型的影响。周期输入的意义在于：输入强度会随时间变化，因此膜电位也会随着输入的强弱变

化而呈现不同的发放节律。

下面给出一个周期电流输入下的 LIF 仿真模板。运行后请同时观察“输入电流曲线”和“膜电位曲线”，并分析它们之间的对应关系。

```
import numpy as np
import matplotlib.pyplot as plt



# 根据 LIF 动力学方程更新一个时间步的膜电位
def lif_step(v_prev, input_current, tau_m, dt, R=1.0, v_rest=0.0):
    return v_prev + dt / tau_m * (-(v_prev - v_rest) + R * input_curren
t)


# 在周期电流输入下仿真 LIF 神经元的膜电位与发放情况
def simulate_lif_periodic(
    base_current=15.0,
    amplitude=5.0,
    cycle_ms=20.0,
    tau_m=10.0,
    v_rest=0.0,
    v_reset=0.0,
    v_th=15.0,
    R=1.0,
    dt=0.1,
    time_window=100.0,
):
    time = np.arange(0, time_window + dt, dt)
    current = base_current + amplitude * np.cos(2 * np.pi * time / cycl
e_ms)
    voltage = np.zeros_like(time)
    spikes = np.zeros_like(time)


    v = v_rest


    for i in range(1, len(time)):
        v = lif_step(v, current[i], tau_m, dt, R, v_rest)
        if v >= v_th:
            spikes[i] = 1
            voltage[i] = v_th
            v = v_reset
        else:
            voltage[i] = v
```

```
    return time, current, voltage, spikes


# 本实验要求修改的参数
base_current = 15.0    # 周期输入电流的基线值
amplitude = 5.0        # 周期输入电流的振幅
cycle_ms = 20.0        # 周期输入电流的周期（ms）
V_th = 15.0            # 发放阈值

# 其余默认参数（一般不需要修改）
tau_m = 10.0           # 膜时间常数
R = 1.0                # 膜电阻
dt = 0.1               # 仿真时间步长（ms）
time_window = 100.0    # 总仿真时长（ms）

time, current, voltage, spikes = simulate_lif_periodic(
    base_current=base_current,
    amplitude=amplitude,
    cycle_ms=cycle_ms,
    tau_m=tau_m,
    v_th=V_th,
    R=R,
    dt=dt,
    time_window=time_window,
)

fig, axes = plt.subplots(2, 1, figsize=(10, 7), sharex=True)

axes[0].plot(time, current, color="tab:orange")
axes[0].set_title("Periodic Input Current")
axes[0].set_ylabel("I(t)")
axes[0].grid(alpha=0.3)

axes[1].plot(time, voltage, color="tab:blue", label="membrane potential
")
axes[1].axhline(V_th, color="tab:red", linestyle="--", label="threshold
")
axes[1].set_title("LIF Membrane Potential under Periodic Input")
```

```
axes[1].set_xlabel("Time (ms)")
axes[1].set_ylabel("V_m (mV)")
axes[1].grid(alpha=0.3)
axes[1].legend()

plt.tight_layout()
plt.show()
```

修改以下参数：

- `base_current`
- `amplitude`
- `cycle_ms`
- `V_th`

在报告中分析以下问题：

1. 输入电流较大的时间段，膜电位上升速度是否更快？
2. 周期电流输入下，神经元发放时刻与输入波峰之间有什么关系？
3. 与恒定电流输入相比，周期输入下的脉冲发放模式有什么不同？

3.3 实验要求

1. 在 `brain` 虚拟环境中安装并导入 `numpy` 和 `matplotlib`。
2. 完成实验说明中的四个主线部分：
    - HH 模型现象观察；
    - LIF 模型在恒定电流输入下的仿真；
    - LIF 理论发放频率与仿真结果对照；
    - LIF 在周期电流输入下的仿真。
3. 补全文中留出的 3 处 `TODO`，保证代码能够正常运行。
4. 至少提交 4 类实验结果图，需覆盖以下内容：

- HH 模型的外加电流与膜电位；
- LIF 恒定电流输入下的膜电位；
- LIF 理论频率与仿真频率对照；
- 周期电流输入下的输入电流与膜电位。
  说明：一类结果可以是单独一个 figure，也可以是同一 figure 中的多个 subplot，只要内容完整即可。

5. 提交实验代码文件。
6. 提交一份简短实验报告或结果说明，报告中必须包含图像、参数设置和对应分析文字，并覆盖前文要求完成的现象观察和问题分析。

实验报告中回答以下问题：

1. HH 模型和 LIF 模型各自更适合描述哪些神经元行为？
2. 在恒定电流输入下，哪些参数会影响 LIF 神经元的发放节律？
3. 为什么当 `RI <= V_th` 时，LIF 神经元不会发放脉冲？
4. 周期输入电流与恒定输入电流相比，对神经元的发放时刻有什么影响？

提交内容如下：

- 实验代码文件，命名为 `exp3_1.py`、`exp3_2.py`、`exp3_3.py`、`exp3_4.py`；
- 一份实验报告；
- 报告中应包含关键图像、参数设置以及对实验现象的分析。